

Examen práctico — ClinicaMobil

Desarrollo de aplicaciones móviles multiplataforma

Estudiante	Jhonsons Plasencia Valdez
Código	202412796
Fecha	16 / 09 / 2026
Duración	4 horas (laboratorio)
Puntaje total	20 puntos
Proyecto de referencia	PharmaMobil (Sesiones 1 a 5)

Instrucciones generales

1. El examen es **individual**. Cualquier código compartido entre estudiantes anula el examen para todos los implicados.
2. Se permite consultar el proyecto **PharmaMobil**, el `TUTORIAL-SESION-05.md` y la documentación oficial de Kotlin, Compose Multiplatform y Koin. **No se permite** copiar el proyecto y renombrarlo: el dominio, las reglas y la paleta son distintos y se revisan.
3. Lee **todo** el enunciado antes de escribir la primera línea. Los anexos contienen las reglas exactas que se califican.
4. La entrega se hace en un **repositorio Git** con la rama `feature/clean-mvvm` (ver sección *Entrega*). Lo que no esté en la rama al cierre del tiempo no se califica.
5. El proyecto **debe compilar**. Un proyecto que no compila se califica solo en la Parte II.
6. Todos los mensajes de validación y de interfaz se toman **literalmente** de los anexos; las pruebas del docente los comparan letra por letra.

El caso

El **Centro de Salud Universitario** atiende cada semestre a miles de estudiantes, docentes y personal administrativo en consultas de medicina general, odontología, psicología y nutrición. Cada consultorio lleva sus propias fichas en papel: cuando un paciente pasa de medicina general a nutrición, el nutricionista no tiene forma de ver el peso registrado la semana anterior, y cuando un médico nuevo se incorpora nadie sabe con certeza qué número de colegiatura tiene registrado.

La dirección del centro ha encargado una aplicación móvil, **ClinicaMobil**, que el personal de admisión usará desde una tableta Android o un iPhone en el mostrador. La primera versión debe permitir **registrar pacientes en el padrón**, **registrar médicos del cuerpo clínico** y dejar preparado el módulo de **historias clínicas** para una versión posterior.

La dirección impone una condición: **los datos se centralizarán en el servidor del área de salud el próximo semestre**, y la aplicación debe permitir cambiar la fuente de datos sin tocar las reglas del negocio ni las pantallas. Además, por tratarse de información de salud, cada regla de validación debe vivir en un único lugar auditable.

Tu tarea es construir ClinicaMobil aplicando **todo** lo desarrollado en PharmaMobil: Kotlin Multiplatform con Compose Multiplatform, arquitectura en capas Clean, MVVM con `StateFlow`, inyección de dependencias con Koin, navegación con menú lateral, identidad visual propia y pruebas automatizadas.

Parte I — Práctica (17 puntos)

Ítem 1 · Proyecto y rama — 0.5 pt

Crea un proyecto Kotlin Multiplatform con Compose Multiplatform para Android e iOS, paquete raíz `pe.edu.upeu.clinicamobil`. Declara en `libs.versions.toml` las dependencias de Koin, iconos Material, Lifecycle y `coroutines-test`. Todo el trabajo va en la rama `feature/clean-mvvm`.

Se califica: compila con `:androidApp:assembleDebug` · cada dependencia de Koin marcada `api` lleva un comentario que justifica por qué no es `implementation`.

Ítem 2 · Capa de dominio — 2.5 pts

En `domain/model` escribe `Paciente`, `Medico`, `HistoriaClinica`, `Atencion` y `EstadoHistoria` según el **Anexo A**, con sus `require` en `init` y sus reglas de negocio propias (`Paciente.esPediatico`, `Atencion.costeo`).

En `domain/repository` escribe las interfaces `PacienteRepository` y `MedicoRepository` con `registrar` y `listar`, ambas `suspend`, documentadas en el lenguaje del negocio (*padrón*, *cuerpo médico*).

Se califica: ninguna clase de `domain` importa Compose, Android, iOS, Ktor ni Koin · las constantes de negocio están en el modelo, no en la pantalla.

Ítem 3 · Casos de uso — 2.5 pts

En `domain/usecase` escribe:

- `RegistrarPacienteUseCase` con `ErroresDePaciente` (cuatro campos) y `PacienteInvalidoException`.
- `RegistrarMedicoUseCase` con `ErroresDeMedico` y `MedicoInvalidoException`.
- `ListarPacientesUseCase` y `ListarMedicosUseCase`.
- El helper `resultadoDe`.

Los casos de uso `Registrar*` reciben **todos los campos como `String`**, aplican **todas** las reglas del Anexo A y devuelven `Result<...>`. El `id` que envían al repositorio es `0L`.

Se califica: mensajes idénticos al Anexo A · `resultadoDe` relanza `CancellationException` · nombre y

DNI se guardan recortados · la especialidad en blanco se guarda como `null` .

Ítem 4 · Capa de datos — 1 pt

En `data/repository` implementa `PacienteRepositorioEnMemoria` y `MedicoRepositorioEnMemoria` . El repositorio **asigna el id** de forma correlativa desde `1` , simula latencia con `delay()` entre 300 y 800 ms y protege la lista con un `Mutex` .

Ítem 5 · Módulo de Pacientes (MVVM) — 2.5 pts

En `presentation/paciente` :

- `PacienteUiState` con `fase` (sealed interface con `Cargando` , `SinPacientes` , `ConPacientes(pacientes: List<PacienteUi>)` , `Error(mensaje)`), `formulario` , `registrando` y `mensajeExito` .
- `FormularioPaciente` con cuatro valores y cuatro errores.
- `PacienteUi` y la extensión `Paciente.aui()` que arma la línea `34 años · 70.5 kg` (**Anexo B**).
- `PacienteViewModel(registrarPaciente, listarPacientes)` con `StateFlow` de solo lectura, `cargarPacientes()` , un `on<Campo>Change` por campo y `registrar()` .
- `PacienteScreen(viewModel, modifier)` que cumple el **RF-03 del Anexo C**.

Se califica: la pantalla no tiene ningún `remember` de estado de negocio · el `when` sobre la fase es exhaustivo · los errores de validación se pintan **sin volver a validar** en el ViewModel · lo escrito sobrevive a la rotación · un doble toque en Registrar no registra dos veces.

Ítem 6 · Módulo de Médicos (MVVM) — 1.5 pts

En `presentation/medico` , el equivalente completo del ítem 5 para `Medico` , cumpliendo el **RF-04**. La especialidad ausente se muestra como `Medicina general` .

Ítem 7 · Inyección de dependencias con Koin — 2 pts

- `di/AppModule.kt` con `dataModule` , `domainModule` , `presentationModule` , `expect` y `val platformModule` e `initKoin()` .
- `actual val platformModule` en `androidMain` y `iosMain` .
- `MainApplication` en `androidApp` declarada en el manifiesto.
- `initKoinIos()` en `iosMain` invocada desde `iOSApp.swift` .
- `App()` dentro de `KoinContext` , pantallas con `koinViewModel()` .

Se califica: los repositorios se registran **por su interfaz** y como `single` · los casos de uso como `factory` · ninguna clase fuera de `di` menciona `*EnMemoria` .

Ítem 8 · Navegación y pantalla de Inicio — 1.5 pts

- `Screen` sealed con `Inicio` , `Pacientes` , `Medicos` , `Historias` .
- Menú lateral y título de la barra superior alimentados por **una sola lista** `DESTINOS` .
- `pantallaActual` sobrevive a la rotación (`rememberSaveable` con `Saver`).

- `InicioScreen(onNavegar: (Screen) -> Unit)` con portada y tres accesos rápidos **navegables**, generados desde una lista de opciones (**RF-01**).
- `Screen.Historias` muestra un `EstadoVacio` (**RF-05**).

Ítem 9 · Identidad visual y componentes — 1 pt

- `ClinicaMobilTheme` con esquemas claro y oscuro **completos** (todos los roles, incluidos `surfaceContainer*`). La paleta **no** puede ser la verde de PharmaMobil ni la morada por defecto.
- Interruptor de modo oscuro al pie del menú.
- Componentes reutilizables `ValidatedTextField`, `MensajeExito` y `EstadoVacio` en `presentation/components`.

Ítem 10 · Pruebas automatizadas — 2 pts

En `shared/src/commonTest`, con `FakePacienteRepository` y `FakeMedicoRepository` (sin `delay`, capaces de fallar a voluntad), escribe las pruebas del **Anexo D**. Mínimo **20 pruebas, 0 fallos**, verificadas con `:shared:testAndroidHostTest`.

Parte II — Teórica (3 puntos)

Responde en `docs/RESPUESTAS.md`, con tus palabras, máximo un párrafo por pregunta. **1 punto cada una.**

1. Cuando los datos se centralicen en el servidor del área de salud, ¿qué archivos de tu proyecto cambian y cuáles quedan intactos? Nómbralos y justifica con la regla de dependencia entre capas.
2. La dirección exige que cada regla de validación viva en **un único lugar auditable**. ¿En qué clase de tu proyecto vive la regla "el DNI debe tener 8 dígitos"? ¿Por qué no debe repetirse en `PacienteScreen` ni en `PacienteViewModel`, y qué pasaría si el modelo `Paciente` la relajara?
3. Explica qué observaría el personal de admisión si `PacienteRepository` estuviera registrado en Koin como `factory` en vez de `single`, y por qué.

Entrega

1. Repositorio Git con la rama `feature/clean-mvvm` y **al menos un commit por ítem**, con mensajes en español, en imperativo.
2. `docs/RESPUESTAS.md` con la Parte II.
3. `docs/CAPTURAS/` con: Inicio (claro y oscuro), Pacientes en sus cuatro fases, Médicos con una especialidad ausente, Historias clínicas.
4. Salida de `./gradlew :shared:testAndroidHostTest` pegada al final de `RESPUESTAS.md`, mostrando el número de pruebas.

Para capturar *Sin pacientes* y *Error* puedes modificar temporalmente `PacienteRepositoryEnMemoria.listar()`; **el commit final no debe contener esa modificación.**

Penalizaciones

Situación	Descuento
<code>presentation</code> importa algo de <code>data</code>	-2
Repositorio registrado por su clase concreta o como <code>factory</code>	-2
Mensaje de validación que no coincide con el Anexo A	-0.5 c/u (máx. -2)
Accesos rápidos de Inicio escritos a mano en vez de generados desde una lista	-1
Paleta de PharmaMobil o morado por defecto	-1
Commit final con el repositorio modificado para forzar estados	-1
Proyecto que no compila	Solo se califica la Parte II

Anexo A — Reglas del negocio

A.1 `Paciente(id: Long, nombre: String, dni: String, edad: Int, peso: Double)`

Campo	Regla	Mensaje del caso de uso
Nombre	Obligatorio (no vacío ni solo espacios)	El nombre es obligatorio
DNI	Obligatorio	El DNI es obligatorio
DNI	Exactamente 8 dígitos (<code>^[0-9]{8}\$</code>)	El DNI debe tener 8 dígitos
Edad	Obligatoria	La edad es obligatoria
Edad	Debe ser un entero	La edad debe ser un número entero
Edad	Entre <code>0</code> y <code>EDAD_MAXIMA</code> (120)	La edad debe estar entre <code>0</code> y <code>120</code>
Peso	Obligatorio	El peso es obligatorio
Peso	Debe ser un número válido (no <code>NaN</code> , no infinito)	El peso debe ser un número válido
Peso	Mayor que cero	El peso debe ser mayor a <code>0</code>

El modelo repite las reglas de rango con `require` en su `init` . Regla propia:

```
/** Un paciente es pediátrico mientras no alcance la mayoría de edad. */
val esPediátrico: Boolean
    get() = edad < EDAD_ADULTO

companion object {
    const val EDAD_ADULTO = 18
    const val EDAD_MAXIMA = 120
}
```

A.2 Medico(id: Long, nombre: String, colegiatura: String, especialidad: String?)

Campo	Regla	Mensaje del caso de uso
Nombre	Obligatorio	El nombre es obligatorio
Colegiatura	Obligatoria	La colegiatura es obligatoria
Colegiatura	Entre 5 y 6 dígitos (^[0-9]{5,6}\$)	La colegiatura debe tener entre 5 y 6 dígitos
Especialidad	Opcional; si viene, al menos 3 caracteres	La especialidad debe tener al menos 3 caracteres

El modelo exige `nombre` y `colegiatura` no vacíos, y que `especialidad` sea `null` o no vacía. El caso de uso convierte la especialidad en blanco a `null`.

A.3 Historias clínicas (solo modelos, sin pantalla)

```
data class Atencion(  
    val medico: Medico,  
    val diagnostico: String,  
    val duracionMinutos: Int  
) {  
    init {  
        require(diagnostico.isNotBlank()) { "El diagnóstico no puede estar vacío" }  
        require(duracionMinutos in 1..DURACION_MAXIMA) { "La atención debe durar entre 1 y 120 minutos" }  
    }  
  
    /** Costo de la atención según el tiempo de consultorio utilizado. */  
    fun costo(): Double = duracionMinutos * TARIFA_POR_MINUTO  
  
    companion object {  
        const val DURACION_MAXIMA = 120  
        const val TARIFA_POR_MINUTO = 2.50  
    }  
}  
  
sealed class EstadoHistoria {  
    data object Abierta : EstadoHistoria()  
    data object EnTratamiento : EstadoHistoria()  
    data object Alta : EstadoHistoria()  
    data class Derivada(val especialidad: String) : EstadoHistoria()  
}  
  
data class HistoriaClinica(  
    val id: Long,  
    val paciente: Paciente,  
    val atenciones: List<Atencion>,  
    val estado: EstadoHistoria  
)
```

Anexo B — Reglas de presentación

Dato	Formato exacto
Línea secundaria del paciente	<edad> años · <peso> kg → 34 años · 70.5 kg ; singular: 1 año · 9.2 kg ; el peso siempre con un decimal (70.0 kg)
Línea secundaria del médico	CMP <colegiatura> · <especialidad> → CMP 12345 · Cardiología
Especialidad ausente	Medicina general
Conteo del padrón	1 paciente / N pacientes
Conteo del cuerpo médico	1 médico / N médicos
Badge pediátrico	Pediátrico (visible solo si esPediatico)
Mensaje de éxito	Paciente "<nombre>" registrado correctamente / Médico "<nombre>" registrado correctamente
Error de carga por defecto	No se pudo cargar el padrón de pacientes / No se pudo cargar el cuerpo médico
Texto del botón mientras registra	Registrando...
Título del módulo de historias	Historias clínicas en construcción

Kotlin común no tiene `String.format`: el decimal del peso se arma a mano, como el formato de soles en PharmaMobil.

Anexo C — Requisitos funcionales

RF-01 · Inicio. Portada con icono `LocalHospital`, nombre `ClinicaMobil` y lema. Encabezado "Qué puedes hacer" y tres accesos rápidos (*Registrar pacientes*, *Registrar médicos*, *Revisar historias clínicas*) que navegan a su módulo; el menú lateral y el título de la barra quedan sincronizados.

RF-02 · Navegación. `ModalNavigationDrawer` con cabecera de marca, cuatro destinos y modo oscuro al pie. La pantalla actual sobrevive a la rotación.

RF-03 · Pacientes. Formulario en tarjeta: Nombre, DNI (teclado numérico), Edad y Peso (estos dos lado a lado; edad con teclado numérico, peso con teclado decimal). Error por campo que desaparece al escribir. Botón Registrar deshabilitado y con texto `Registrando...` durante la operación. Tras registrar: formulario vacío, `MensajeExit`, padrón recargado. Cuatro fases excluyentes: *Cargando* (indicador + texto), *Sin pacientes* (`EstadoVacio`), *Con pacientes* (tarjetas con nombre, DNI, línea secundaria y badge), *Error* (`EstadoVacio` en color de error + botón `Reintentar`). Encabezado con conteo. El formulario sobrevive a la rotación.

RF-04 · Médicos. Igual que RF-03 con Nombre, Colegiatura (teclado numérico) y Especialidad opcional. Especialidad ausente: `Medicina general`.

RF-05 · Historias clínicas. `EstadoVacio` con icono `Assignment`, título `Historias clínicas` en construcción y descripción.

RF-06 · Plataformas. Compila y arranca en Android; `:shared:compileKotlinIosSimulatorArm64` compila; `iosApp.swift` inicia Koin.

Anexo D — Pruebas mínimas

Clase	Casos obligatorios
<code>PacienteTest</code>	Rechaza nombre vacío · rechaza edad fuera de rango · <code>esPediatico</code> cierto con 17 y falso con 18
<code>AtencionTest</code>	Rechaza 0 minutos · rechaza 121 minutos · <code>costo()</code> de 30 minutos es <code>75.0</code>
<code>RegistrarPacienteUseCaseTest</code>	Acepta un paciente válido · al menos cinco mensajes del Anexo A.1 · el id lo asigna el repositorio · el fallo del repositorio llega como <code>Result.failure</code>
<code>RegistrarMedicoUseCaseTest</code>	Colegiatura de 4 dígitos · especialidad de 2 caracteres · especialidad en blanco se guarda como <code>null</code>
<code>PacienteRepositorioEnMemoriaTest</code>	Ids correlativos · <code>listar</code> en orden de registro
<code>PacienteViewModelTest</code>	Arranca en <code>SinPacientes</code> · muestra <code>34 años</code> · <code>70.5 kg</code> · pasa a <code>Error</code> si el repositorio falla · errores de validación caen en el formulario, no en la fase · registrar limpia el formulario y recarga
<code>AppModuleTest</code>	Resuelve <code>PacienteRepository</code> como <code>PacienteRepositorioEnMemoria</code> · el repositorio es único · resuelve los cuatro casos de uso

Los `ViewModelTest` instalan `Dispatchers.setMain(UnconfinedTestDispatcher())` antes de cada caso y lo restauran después.

Anexo E — Correspondencia con PharmaMobil

PharmaMobil	ClinicaMobil
<code>pe.edu.upeu.pharmamobil</code>	<code>pe.edu.upeu.clinicamobil</code>
Producto / <code>STOCK_MINIMO = 10</code> / "Reponer"	Paciente / <code>EDAD_ADULTO = 18</code> / "Pediátrico"
Cliente (teléfono opcional)	Medico (especialidad opcional)
Pedido , <code>DetallePedido</code> , <code>EstadoPedido</code>	<code>HistoriaClinica</code> , <code>Atencion</code> , <code>EstadoHistoria</code>

PharmaMobil	ClinicaMobil
Screen.Productos / Clientes / Pedidos	Screen.Pacientes / Medicos / Historias
Icons.Default.LocalPharmacy / Medication / Person / ShoppingCart	Icons.Default.LocalHospital / Person / MedicalServices / Assignment
Formato S/ 12.50	Formato 34 años • 70.5 kg
PharmaMobilTheme verde	ClinicaMobilTheme con paleta propia

Resumen de puntaje

Parte	Ítem	Puntos
I	1 · Proyecto y rama	0.5
I	2 · Dominio	2.5
I	3 · Casos de uso	2.5
I	4 · Datos	1
I	5 · Módulo de Pacientes	2.5
I	6 · Módulo de Médicos	1.5
I	7 · Koin	2
I	8 · Navegación e Inicio	1.5
I	9 · Identidad visual y componentes	1
I	10 · Pruebas	2
II	Preguntas 1 a 3	3
	Total	20

Anexo de entrega

Respuestas de la Parte II, evidencias de funcionalidades y resultado de pruebas

Jhonsons Plasencia Valdez · Código 202412796

Repositorio: `github.com/Plasencia2007/ClinicaMobil-simulacro`
Rama: `feature/clean-mvvm`

Parte II — Respuestas

1. Al centralizar los datos en el servidor, ¿qué archivos cambian y cuáles quedan intactos?

Solo cambian dos cosas: se crea una nueva clase en `data/repository` (por ejemplo `PacienteRepositorioRemoto.kt` y `MedicoRepositorioRemoto.kt`) que implemente las mismas interfaces `PacienteRepository`/`MedicoRepository` consumiendo Ktor u otro cliente HTTP, y se cambia una sola línea en `di/AppModule.kt` (el `single<PacienteRepository>` { `PacienteRepositorioEnMemoria()` } pasa a apuntar a la implementación remota). Todo lo demás queda intacto: `domain/model`, `domain/repository`, `domain/usecase` y toda la capa `presentation`. Esto es la regla de dependencia de Clean Architecture: el dominio no depende de nada externo y las capas externas dependen hacia adentro; como los casos de uso y los ViewModels solo conocen la interfaz, sustituir la fuente de datos es un cambio contenido en la capa de datos y en el punto único de ensamblado (Koin).

2. ¿En qué clase vive la regla "el DNI debe tener 8 dígitos"?

Vive de forma auditable en `RegistrarPacienteUseCase`, que decide si un DNI es válido y qué mensaje mostrar; `Paciente.init` repite la condición como invariante estructural de último nivel. No debe repetirse en `PacienteScreen` ni en `PacienteViewModel` porque ninguno de los dos vuelve a validar: la pantalla solo pinta el error que ya viene en el formulario, y el ViewModel solo reenvía los errores del caso de uso. Repetirla crearía dos fuentes de verdad que podrían desincronizarse. Si el modelo relajara su `require`, el registro normal seguiría bloqueando DNIs inválidos porque el caso de uso valida antes de construir el Paciente; pero cualquier otro punto que construya un Paciente directamente (una semilla de datos, una prueba, una futura importación) podría colar un DNI inválido sin que nadie lo note.

3. ¿Qué observaría el personal de admisión si `PacienteRepository` fuera `factory` en vez de `single`?

Cada vez que Koin resuelva `PacienteRepository` —por ejemplo, cada vez que se crea un nuevo `PacienteViewModel` al volver a entrar a `Pacientes`— con `factory` se construiría una instancia nueva de `PacienteRepositorioEnMemoria`, con su lista vacía y el contador de id reiniciado en 1. El personal vería que el padrón "se vacía" cada vez que navegan a otra pantalla y regresan: los pacientes recién registrados desaparecerían, porque ya no es el mismo objeto en memoria sino una copia nueva sin datos. Con `single`, Koin construye una única instancia para toda la vida de la app, así que el padrón persiste mientras la app siga abierta.

Salida de pruebas — ./gradlew :shared:testAndroidHostTest

30 pruebas ejecutadas (28 nuevas del Reto 02 + 2 del scaffold inicial), **0 fallos**:

```
PacienteTest tests=4 failures=0 errors=0
AtencionTest tests=3 failures=0 errors=0
RegistrarPacienteUseCaseTest tests=8 failures=0 errors=0
RegistrarMedicoUseCaseTest tests=3 failures=0 errors=0
PacienteRepositorioEnMemoriaTest tests=2 failures=0 errors=0
PacienteViewModelTest tests=5 failures=0 errors=0
AppModuleTest tests=3 failures=0 errors=0
SharedCommonTest tests=1 failures=0 errors=0
SharedLogicAndroidHostTest tests=1 failures=0 errors=0
```

BUILD SUCCESSFUL

Evidencias — Inicio y navegación



Tema claro



Tema oscuro

Evidencias — Módulo de Pacientes

7:40

Pacientes

Registro de paciente

Nombre

DNI

Edad

Peso

Registrar

Padrón de pacientes

0 pacientes

Sin pacientes

Aún no hay pacientes registrados en el padrón.

Inicio

Pacientes

Médicos

Historias

Fase «Sin pacientes»: formulario listo, padrón vacío.

7:41

Pacientes

Registro de paciente

Nombre

Ana Torres

DNI

12345678

Edad

34

Peso

70.5

Registrando...

Padrón de pacientes

0 pacientes

Sin pacientes

Aún no hay pacientes registrados en el padrón

Inicio

Pacientes

Médicos

Historial

Registro en curso: botón deshabilitado, texto «Registrando...».

7:42 



Pacientes



Registro de paciente

Nombre

DNI

Edad

Peso

Registrar



Paciente "Ana Torres" registrado correctamente

Padrón de pacientes

1 paciente

A

Ana Torres

DNI 12345678

34 años · 70.5 kg



Inicio

Pacientes

Médicos

Historial

Fase «Con pacientes»: mensaje de éxito y tarjeta «34 años · 70.5 kg».

Evidencias — Módulo de Médicos

7:43 



Médicos

Registro de médico

Nombre

Colegiatura

Especialidad (opcional)

Registrar



Médico "Carlos Ramos" registrado correctamente

Cuerpo médico

1 médico

C

Carlos Ramos

CMP 54321 · Medicina general





Inicio



Pacientes



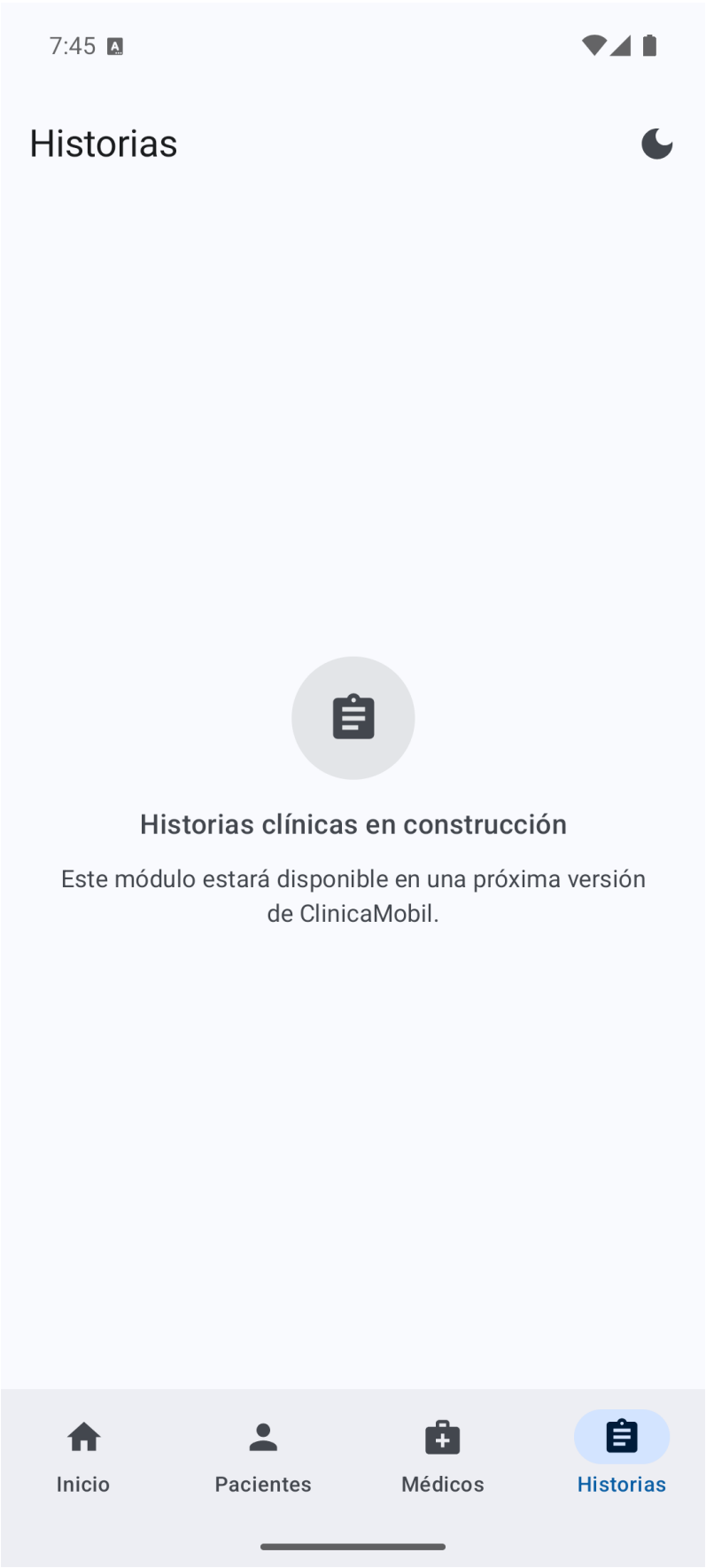
Médicos



Historial

Médico sin especialidad: «CMP 54321 · Medicina general».

Evidencias — Historias clínicas



Pantalla de Historias clínicas: módulo en construcción (RF-05).